

인터뷰 키트 · 주문·재고 API v2

제출 [@ 4c595e36efb5489dd0234d834cfdb583048ccfc1](https://github.com/wanted-hack-inchyangv/taeyun-order-api)

기준 버전 v2-be07fb44

워크벤치 <https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?tab=questions>

질문 11개 · 필수 3개 · 권장 4개 · 선택 4개 · 기본 질문 5개

LLM이 쓴 문장 중 5개가 질문 검사에 걸려 기본 질문으로 바꿨습니다.

이력서 연결 질문에는 지원자 이력서의 인용이 들어 있습니다. 복사한 내용은 면접 담당자에게만 공유해 주세요.

45분 진행안 구간 합 44분

도입 5분	
실패 디브리핑 16분	Q1 필수 · 실패 디브리핑 · 8분 Q2 필수 · 실패 디브리핑 · 8분
설계 트레이드오프 6분	Q4 권장 · 설계 트레이드오프 · 6분
강점 확인 12분	Q3 필수 · 강점 확인 · 6분 Q5 권장 · 강점 확인 · 6분
지원자 질문 5분	

60분 진행안 구간 합 55분

도입 5분	
실패 디브리핑 16분	Q1 필수 · 실패 디브리핑 · 8분 Q2 필수 · 실패 디브리핑 · 8분
설계 트레이드오프 6분	Q4 권장 · 설계 트레이드오프 · 6분
강점 확인 12분	Q3 필수 · 강점 확인 · 6분 Q5 권장 · 강점 확인 · 6분
요구사항 확장 6분	Q6 권장 · 요구사항 확장 · 6분
이력서 연결 5분	Q7 권장 · 이력서 연결 · 5분
지원자 질문 5분	

Q1 필수 실패 디브리핑 원인 분석 · 8분

R-06 실행 기록에서 같은 키로 다른 본문을 보낸 두 요청이 모두 201로 처리됐습니다. 주문 생성 경로에서 키와 본문을 비교하는 판단이 어디에서 이뤄지고 있는지 설명해 주시겠어요?

의도 먹등 키 충돌을 걸러내는 판단 지점을 지원자가 코드에서 어떻게 짚어내는지 확인한다.

꼬리 질문

1. 같은 키로 들어온 두 번째 요청이 처리되는 순서를 코드 흐름대로 말씀해 주시겠어요?
2. 키에 저장된 본문과 새 본문을 비교할 때 무엇을 기준으로 같다·다르다를 정하나요?
3. 충돌을 거절로 바꿀 때 재고 차감과 주문 저장 중 어느 시점에 검사를 두시겠어요?

좋은 답변의 신호

- 키 조회와 본문 비교가 어느 단계에서 일어나는지 코드 위치를 들어 설명한다.
- 충돌 판정을 재고 차감 이전에 두어야 하는 이유를 부작용 관점에서 말한다.

우려 신호

- 키 존재 여부만 확인하고 본문 비교가 없다는 점을 짚지 못한다.
- 충돌 응답을 만들 위치를 말하면서 기존 주문·재고에 미치는 영향을 언급하지 않는다.

근거

R-06 · 먹등 키 충돌

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-06>

R-06 재생

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-06&run=db787431-adaf-49df-b3ac-d4f7e839fa97>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

src/routes/order-routes.ts:30-33

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L30-L33>

R-07 실행 기록에서 같은 키의 동시 요청 10건 중 5건만 201이고 응답 id도 서로 달랐으며 재고가 0까지 줄었습니다. 같은 키의 요청들이 겹칠 때 어떤 순서로 처리되는지 설명해 주시겠어요?

의도 동시 요청에서 키 기록과 재고 차감이 어떤 순서로 엮히는지 지원자가 분석하는지 확인한다.

꼬리 질문

1. 두 요청이 거의 동시에 들어오면 키 조회와 키 저장 사이에 어떤 일이 생기나요?
2. 재고 차감이 여러 번 일어난 경로를 실행 기록의 숫자로 어떻게 설명하시겠어요?
3. 동시성을 다루는 방식을 바꿀 때 처리량과 정합성 중 무엇을 기준으로 정하시겠어요?

좋은 답변의 신호

- 키 조회와 저장 사이의 경합 구간을 짚고 재고 차감 횟수와 연결해 설명한다.
- 동시성 제어를 도입할 때 비용과 정합성의 맞바꿈을 언급한다.

우려 신호

- 동시 요청을 순차 요청과 같은 상황으로 설명한다.
- 재고가 0이 된 이유를 키 처리와 연결하지 못한다.

근거

R-07 · 같은 키 동시 요청

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-07>

R-07 재생

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-07&run=7c641beb-fb34-429f-b4da-ccc344aacd15>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

R-03 재고 부족 거절 기준은 통과했습니다. 이 요구사항을 보장하는 코드가 어떻게 동작하는지 설명해 주시겠어요?

의도 통과한 R-03 재고 부족 거절 기준을 지원자가 직접 설명할 수 있는지, 경계 조건에서 깨지는 조건까지 알고 있는지 확인한다.

꼬리 질문

1. 이 경우를 처리하는 코드는 어느 파일과 함수에 있나요?
2. 그 처리가 올바르게 동작하는 원리는 무엇인가요?
3. 다른 구현 방식과 비교하면 지금 방식의 장단점은 무엇인가요?

좋은 답변의 신호

- 처리 코드의 위치를 스스로 찾고 동작 순서를 정확히 설명한다.
- 이 처리가 깨지는 입력이나 동시 요청 조건을 스스로 말한다.

우려 신호

- 통과한 결과만 말하고 코드의 동작 원리를 설명하지 않는다.
- 다른 구현 방식과의 차이를 구체적으로 비교하지 않는다.

근거

R-03 · 재고 부족 거절

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-03>

R-03 재생

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-03&run=ce8791ed-2fa3-4d3c-8fa0-1e495b5efdd6>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

Q4 **권장** 설계 트레이드오프 설계와 변경 용이성 · 6분

가장 긴 함수가 라우트 파일의 `createOrderRoutes`로 34줄이라는 신호가 있습니다. 이 함수가 맡은 일을 나열해 주시고, 그중 어디까지를 라우트 계층에 두기로 정했는지 설명해 주시겠어요?

의도 라우트 함수에 섞인 책임을 지원자가 어떻게 구분하고 경계를 정하는지 확인한다.

꼬리 질문

1. 이 함수 안에서 요청 파싱과 도메인 판단이 각각 어디에서 이뤄지나요?
2. 도메인 규칙을 별도 모듈로 옮긴다면 어떤 함수가 남고 무엇이 빠지나요?
3. 계층을 나눌 때 얻는 것과 잃는 것을 어떤 기준으로 저울질하시겠어요?

좋은 답변의 신호

- 함수 안의 단계를 나열하고 각 단계의 책임을 구분해 말한다.
- 분리했을 때의 이득과 비용을 구체적인 변경 시나리오로 설명한다.

우려 신호

- 함수 길이를 줄이는 것 자체를 목표로 말한다.
- 요청·응답 객체에 도메인 규칙이 의존하는지 여부를 언급하지 않는다.

근거

`src/routes/order-routes.ts:8-41`

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L8-L41>

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-12>

R-04 입력 검증 기준은 통과했습니다. 이 요구사항을 보장하는 코드가 어떻게 동작하는지 설명해 주시겠어요?

의도 통과한 R-04 입력 검증 기준을 지원자가 직접 설명할 수 있는지, 경계 조건에서 깨지는 조건까지 알고 있는지 확인한다.

꼬리 질문

1. 이 경우를 처리하는 코드는 어느 파일과 함수에 있나요?
2. 그 처리가 올바르게 동작하는 원리는 무엇인가요?
3. 다른 구현 방식과 비교하면 지금 방식의 장단점은 무엇인가요?

좋은 답변의 신호

- 처리 코드의 위치를 스스로 찾고 동작 순서를 정확히 설명한다.
- 이 처리가 깨지는 입력이나 동시 요청 조건을 스스로 말한다.

우려 신호

- 통과한 결과만 말하고 코드의 동작 원리를 설명하지 않는다.
- 다른 구현 방식과의 차이를 구체적으로 비교하지 않는다.

근거**R-04 · 입력 검증**

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-04>

R-04 재생

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-04&run=d0198e58-0399-4e79-8c8a-d53be52066df>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

같은 서비스를 여러 인스턴스로 띄워 요청이 서로 다른 인스턴스로 들어가는 상황을 가정해 보겠습니다. 지금 구조에서 멍등성과 재고 처리가 어떻게 달라질지 설명해 주시겠어요?

의도 인메모리 상태를 전제한 설계가 다중 인스턴스에서 어떤 한계를 갖는지 지원자가 인식하는지 확인한다.

꼬리 질문

1. 키 기록이 인스턴스마다 따로 존재할 때 같은 키의 두 요청은 어떻게 처리되나요?
2. 재고 차감이 여러 인스턴스에 나뉘면 어떤 값이 관측될 수 있나요?
3. 여러 인스턴스로 확장할 때 정합성과 응답 지연 중 무엇을 우선할지 어떤 기준으로 정하시겠어요?

좋은 답변의 신호

- 인스턴스별 상태 분리가 멍등성과 재고에 미치는 영향을 구체적으로 말한다.
- 확장 시 필요한 공유 지점을 짚고 그에 따르는 비용을 언급한다.

우려 신호

- 인메모리 상태가 인스턴스마다 다르다는 점을 언급하지 않는다.
- 확장을 코드 수정 없이 가능하다고 단정한다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-01>

R-05 · 멍등 재전송

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-05>

R-06 · 멍등 키 충돌

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-06>

Q7 권장 이력서 연결 트레이드오프 판단과 확장 · 5분 기본 질문

이 경험에서 본인이 맡은 범위와 가장 어려웠던 기술적 결정을 설명해 주시겠어요?

이력서: 결제 웹훅(payment webhook) 수신을 맡아 같은 웹훅이 재전송돼도 결제가 중복 반영되지 않도록 멍등(idempotent) 처리를 담당했습니다.

의도 이력서에 적힌 경험에서 지원자가 직접 맡은 범위와 기술적 판단의 근거를 확인한다.

꼬리 질문

1. 그 작업에서 본인이 직접 설계하거나 구현한 부분은 어디까지였나요?
2. 그 결정을 내릴 때 어떤 기준으로 대안을 비교하셨나요?
3. 지금 다시 한다면 어떤 부분을 다르게 선택하시겠어요?

좋은 답변의 신호

- 본인이 맡은 범위와 팀이 맡은 범위를 구분해 설명한다
- 결정의 근거로 당시의 제약 조건과 비교한 대안을 구체적으로 든다

우려 신호

- 맡은 범위를 물어도 팀 전체의 결과만 설명한다
- 결정의 이유를 물으면 기술 이름만 나열하고 적용 조건을 말하지 않는다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?tab=resume#claim-28cd7ebc-8a1c-4e0f-8953-9efdc4d196f1>

R-05 · 멍등 재전송

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-05>

선택 질문 4개 · 22분

Q8 선택 설계 트레이드오프 설계와 변경 용이성 · 6분 기본 질문

R-12 설계 검토 항목을 함께 보겠습니다. 모듈과 책임을 나눈 기준은 무엇이었나요?

의도 R-12 설계·변경 용이성 항목에서 구조를 정한 기준과 변경 비용에 대한 판단을 확인한다.

꼬리 질문

1. 이 코드가 맡고 있는 책임을 나누어 설명해 주시겠어요?
2. 요구사항이 하나 바뀌면 이 구조에서 어느 부분을 고치게 되나요?
3. 구조를 더 나누면 얻는 것과 잃는 것은 무엇인가요?

좋은 답변의 신호

- 코드의 책임을 함수와 파일 단위로 나누어 설명한다.
- 구조를 바꿀 때의 비용과 이득을 요구사항 변경 예시로 비교한다.

우려 신호

- 구조를 정한 이유를 설명하지 않고 선호로만 답한다.
- 변경 예시를 들 때 영향 범위를 짚지 않는다.

근거

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-12>

Q9 선택 요구사항 확장 트레이드오프 판단과 확장 · 6분 기본 질문

프로세스를 다시 시작해도 주문과 상태가 남아야 하는 상황을 가정해 보겠습니다. 지금 구현에서 가장 먼저 바뀌어야 할 부분은 어디인가요?

의도 요구사항이 바뀌었을 때 현재 구현(R-01·R-02·R-05)에서 영향을 받는 부분과 필요한 보장을 판단하는지 확인한다.

꼬리 질문

1. 지금 구현에서 그 상황의 영향을 받는 상태는 무엇인가요?
2. 그 상태를 안전하게 다루려면 어떤 보장이 필요한가요?
3. 그 보장을 얻는 방법들 가운데 어떤 기준으로 하나를 고르시겠어요?

좋은 답변의 신호

- 영향을 받는 상태와 코드 위치를 구체적으로 짚는다.
- 선택지마다 비용과 보장 수준을 비교해 하나를 고른다.

우려 신호

- 특정 기술 이름만 말하고 필요한 보장을 설명하지 않는다.
- 현재 구현이 받는 영향을 구체적으로 짚지 않는다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-01>

R-02 · 상품·주문 조회

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-02>

R-05 · 먹등 재전송

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-05>

Q10 선택 이력서 연결 요구사항 이해와 구현 정확성 · 5분

회사 예약 API에서 같은 시간대 중복 예약을 막는 검증 로직은 어떤 조건으로 판정하도록 구현하셨나요?

이력서: 독서 모임 예약(booking) API를 개발하며 같은 시간대에 같은 장소·모임이 중복 예약되지 않도록 검증 로직을 구현했습니다.

의도 이력서의 중복 예약 검증이 어떤 경계 조건과 저장 구조를 전제로 했는지 확인한다.

꼬리 질문

1. 겹침 판정은 시작·종료 시각을 어떤 비교식으로 처리했나요?
2. 검증과 저장 사이에 다른 요청이 끼어들면 어떻게 되나요?
3. 개인 프로젝트의 인메모리 저장 방식과 회사 구현은 어떤 차이가 있었나요?

좋은 답변의 신호

- 겹침 판정의 경계 조건(끝점 포함 여부)을 명확히 설명한다
- 검증 시점과 저장 시점의 순서, 트랜잭션 범위를 설명한다
- 회사 구현과 개인 프로젝트의 저장 구조 차이를 구분해 설명한다

우려 신호

- 겹침 판정을 '시간이 겹치면 거절'로만 설명한다
- 검증과 저장 사이의 경합 가능성을 고려하지 않는다
- 경계 조건(끝점)을 설명하지 못한다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?tab=resume#claim-7ddc9e41-a4d9-48f4-a4bd-f1087c68088b>

R-03 · 재고 부족 거절

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-03>

라우팅·서비스·저장소 계층을 분리한 리팩터링에서 계층 간 경계는 어떤 기준으로 나누셨나요?

이력서: 모임·예약·결제 도메인의 라우팅(routes)·서비스(services)·저장소(repository) 계층을 분리하는 리팩터링에 참여해 신규 기능 추가 시 변경 범위를 줄였습니다.

의도 이력서의 계층 분리 경험이 실제로 어떤 책임 분담과 변경 범위 축소로 이어졌는지 확인한다.

꼬리 질문

1. 각 계층이 책임지는 것과 책임지지 않는 것은 무엇이었나요?
2. 신규 기능을 추가할 때 실제로 몇 개 계층을 수정하게 되었나요?
3. 계층 분리 이후 테스트는 어느 계층에 두었나요?

좋은 답변의 신호

- 계층별 책임과 의존 방향을 구체적으로 설명한다
- 리팩터링 전후 변경 범위를 사례와 함께 설명한다
- 테스트 배치 기준을 계층과 연결해 설명한다

우려 신호

- 계층을 '폴더로 나눴다'는 수준으로만 설명한다
- 변경 범위가 줄었는지 사례를 들지 못한다
- 계층 간 의존 방향을 설명하지 못한다

근거**이력서 연결**

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?tab=resume#claim-089bb6b7-f96e-472b-9589-582c334d8fca>

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-12>

평가 척도 (역량별 4단계)

면접관이 기입할 때 쓰는 기준이며 시스템은 값을 채우지 않습니다.

요구사항 이해와 구현 정확성

명세의 동작 조건을 빠짐없이 이해하고 그대로 구현한다.

- | | |
|----------------|--|
| 1 미흡 | 명세의 필수 동작을 설명하지 못하거나, 구현과 명세가 다르다는 점을 알아차리지 못한다. |
| 2 보완 필요 | 명세의 주요 동작은 설명하지만, 실패한 요구사항이 어느 조건을 어겼는지 스스로 짚지 못한다. |
| 3 충족 | 요구사항마다 입력과 출력 조건을 설명하고, 실패한 요구사항이 어느 조건을 어겼는지 짚는다. |
| 4 탁월 | 명세의 모호한 부분을 먼저 찾아 가정을 밝히고, 그 가정이 구현과 테스트에 어떻게 반영되었는지 설명한다. |

경계 조건과 실패 처리

잘못된 입력과 자원 부족 같은 실패 경로를 예측하고 일관되게 처리한다.

- | | |
|----------------|---|
| 1 미흡 | 잘못된 입력이나 자원 부족 같은 실패 경로를 떠올리지 못한다. |
| 2 보완 필요 | 실패 경로를 물으면 떠올리지만, 처리 방식과 응답 형식을 구체적으로 말하지 못한다. |
| 3 충족 | 경계값과 실패 경로를 스스로 나열하고, 각각의 응답 코드와 상태 변화를 설명한다. |
| 4 탁월 | 재시도 중 실패나 부분 실패처럼 실패가 겹치는 상황까지 다루고, 상태가 일관되게 남는 근거를 설명한다. |

설계와 변경 용이성

책임을 나누어 요구가 바뀌었을 때 고칠 곳이 좁고 분명한 구조를 만든다.

- 1 미흡 코드 구조를 그렇게 나눈 이유를 설명하지 못한다.
- 2 보완 필요 구조는 설명하지만, 요구가 바뀌면 어디를 고쳐야 하는지 분명히 말하지 못한다.
- 3 충족 책임을 나눈 기준을 설명하고, 요구 변경 하나에 대해 고칠 위치를 코드에서 짚는다.
- 4 탁월 현재 구조의 한계를 스스로 짚고, 다른 구조와 비교해 선택에 따르는 비용을 설명한다.

원인 분석 면접에서만 확인

관측된 실패에서 근거를 따라 원인 코드까지 범위를 좁힌다.

- 1 미흡 재생 기록을 보고도 실패 원인을 추정하지 못한다.
- 2 보완 필요 원인 후보를 말하지만, 근거를 대지 못하고 추측에 머문다.
- 3 충족 재생 기록의 기대값·실제값에서 원인 코드 위치까지 스스로 좁힌다.
- 4 탁월 원인을 좁힌 뒤, 같은 원인으로 생길 수 있는 다른 결함과 재발을 막는 방법까지 제시한다.

트레이드오프 판단과 확장 면접에서만 확인

선택지의 비용과 이점을 기준을 세워 비교하고 조건이 바뀌면 판단을 조정한다.

- 1 미흡 선택지 사이의 차이를 설명하지 못하고 한 가지 방법만 제시한다.
- 2 보완 필요 선택지를 나열하지만, 무엇을 기준으로 골라야 하는지 말하지 못한다.
- 3 충족 정합성, 성능, 복잡도 같은 판단 기준을 밝히고, 과제 조건에서 그 방법을 고른 이유를 설명한다.
- 4 탁월 트래픽 증가나 다중 인스턴스처럼 조건이 바뀌면 선택이 어떻게 달라지는지 근거와 함께 설명한다.